The Orchestrator = Head Chef

9 AI agents is easy. Making them run in the right order, some together, loop back when unsafe, save after each step, and run once even when the browser asks 20 times — that is the orchestrator.

The kitchen picture

9 cooks, each good at one thing: one chops, one fries, one plates, one tastes salt, one checks allergies. Who says when each works? Who runs the salad cook and soup cook together? Who sends a dish back if it is too salty? The head chef. In code that is the orchestrator.

The one sentence to remember

Agents = the brains.
Orchestrator = the pipes + the rules.
Most hard work is in the pipes, NOT the prompts.

What it actually does

Job	What it means
Order	Pick which agent runs when.
Parallel	Run agents that don't need each other together.
Data pass	Give Agent 3's output to Agent 6.
Looping	If safety complains, send work back to redo.
State	Remember what finished (survive refresh).
Errors	On a fail: skip, stop, or continue?
Once-only	Never let the whole thing run twice.

In MediIQ the orchestrator is one plain TypeScript file: `lib/agents/pipeline.ts`. No framework.

Built by Hand — not LangGraph

Ready-made tools exist: LangGraph, LangChain, CrewAI, AutoGen. MediIQ uses none of them. Proof: the only imports in `pipeline.ts` are the database and its own agent files. Zero frameworks.

Why build it by hand?

- Flow was simple: one fan-out, one straight line, one loop. A full graph tool is too heavy for that.
- Full control of saving: each result must land in an exact MongoDB field (dot-notation) so nothing gets overwritten.
- Serverless timing: Vercel kills background work the instant a reply is sent — needed direct control.
- Fewer dependencies: one readable file, nothing extra to break.

Say this in an interview

"I wrote the orchestration myself. 9-agent pipeline with a verify-and-loop cycle, parallel fan-out, and an atomic once-only claim. I looked at LangGraph but my flow was mostly linear with one loop, and I wanted control over saving and serverless timing — so a framework would just be overhead." Shows a **decision**, not ignorance.

The Five Files

File	Job (think of it as...)
types.ts	Shape every agent must follow. Job description.
registry.ts	Info on all 9 agents. Org chart on the wall.
index.ts	Number → function. Phone book.
llm.ts	Picks which AI model answers. Staffing manager.
pipeline.ts	The orchestrator. Head chef.

Plus 2 API routes that *start* it: `run-precall` and `run-pipeline`.

The Agent Contract

Before you can boss agents around, they all must look the same from outside. A contract = a rule: "whatever you do inside, you take input like *this* and give output like *this*."

Why this one idea helps

- Swappable: each agent is a black box with the same plug. Orchestrator calls all 9 the same way.
- 3-state status: not just pass/fail. **flag** = "I worked but found a problem" — this powers the safety loop.
- Self-documenting: `logs` + `durationMs` come back every time, so admin tools show what happened and how slow.
- Self-describing: `feedsInto` / `triggeredBy` tell each result its place in the pipeline.

The 3 statuses

ok = worked fine.
flag = worked, but found a problem (e.g. Agent 7 spots a dangerous drug).
error = could not do the job at all.

The Registry (written map)

`registry.ts` runs nothing. It *describes* the system: for each agent — what it needs, what it makes, who it feeds, what data it touches. One entry (Agent 7) even says `feedsInto: [6, 9]` — it feeds BACK into 6. That is the loop.

Why write a registry?

- Powers the admin "Full Pipeline Flow" screen — the graph is drawn from this data, not hard-coded.
- Documents the design in code, so it can't go stale like a wiki page.
- Reading `feedsInto` across all 9 literally shows the graph — and the one cycle (7→6, 8→6).

The four phases

Phase	Agents	Job
gather	1,2,3,4,5	Collect all the facts.
prescribe	6	Make the decision.
verify	7,8	Check the decision is safe.
detect	9	Record anything new learned.

The Agent Map + Model Layer

`index.ts` maps a number to a function, so you can run "agent N" without an `if` for every N. This is **dynamic dispatch**. `PIPELINE_ORDER = [1..9]`. Admin tools use it to run any single agent for testing. Agents never call the AI directly — they call `llm.ts`, which picks the model. That is orchestrating *models* instead of agents.

Two Start Points + Serverless Bug

Route	When / runs
run-precall	Patient locks details → agents 3,4,5 in parallel.
run-pipeline	Doctor page loads → agents 2,6,7,8,9.

The serverless bug

Vercel freezes a function the instant it sends its reply. So "reply now, run agents in background" (`setImmediate`) gets KILLED — pipeline never runs, doctor's page polls forever showing "Waiting".

Fix: `await` the whole pipeline inside the route (function stays alive) + `maxDuration=300` (ask Vercel for up to 300s; free tier caps at 60s).

Run Exactly Once — Atomic Claim

The doctor's page polls every 2.5s, and can be open in 2 tabs. If each poll started the pipeline you'd get many copies: wasted money, corrupted data, duplicate prescriptions.

The fix

Before any work, the route tries to **claim** the job: `findOneAndUpdate` with a filter on the current state and a `$set` to the next state. Only one caller matches — everyone else gets `null` and returns "already running". **Atomic** = the read and write happen as one step, so no two requests can both win.

State Machine — One Field

All the orchestrator's memory is one MongoDB field: `agentPipelineStep`. A simple string saying where the pipeline is right now.

- Survives everything: restart, refresh, closed laptop — state is in the DB, not memory. Reopen and it resumes.
- Drives the UI free: the progress bar just reads this string.
- Enables the claim: one field can be compare-and-swapped atomically.
- Instantly debuggable: one look shows where a stuck pipeline stopped.

Subtle bug worth telling

Slow pre-call agents once finished *after* the call ended, and their "done" write reset the state from `call_ended` back to `agents_345_done` — so the post-call pipeline never fired and the page hung. **Fix:** make that write *conditional* — only advance if still in the pre-call state.

Phase 1 — Parallel Fan-out

Agents 3, 4, 5 don't need each other (past cases / medical facts / allergies). Running them one by one wastes time, so they are started together (fanned out).

allSettled, not all

Method	Behaviour
Promise.all	If ANY one fails, the whole thing throws. The two that succeeded are thrown away.
Promise.allSettle	Waits for all 3 no matter what, tells you each one: done or failed. Nothing lost.

Real reason: Agent 4 calls government APIs that sometimes time out. With `all`, one slow API would throw away Agent 3 and 5's finished work.

Speed win

- Times 25s, 67s, 12s.
- In sequence = 25+67+12 = **104s**.
- In parallel = the slowest = **67s**.
- It runs *during* the video call, so the patient waits about **0s**.

Phase 2 — Sequential Chain

After the call, the rest must run in order because each needs the one before. This is a dependency chain: Agent 2 (transcript) → Agent 6 (prescribe, needs 2,3,4,5) → Agents 7 + 8 (check, both need 6) → Agent 9 (findings).

Self-healing step

If the call was short and agents 3/4/5 weren't done, Agent 6 would get empty context and prescribe badly. So before prescribing the orchestrator checks and repairs:

1. Re-read the DB first — the pre-warm may have JUST finished; adopt that work.
2. Run ONLY the ones still truly missing — never redo expensive work.
3. Still parallel + `allSettled` — the repair uses the same safe pattern.

Verify-and-Loop Cycle

This is the most-asked part. It is what makes the pipeline a **graph with a cycle**, not a straight line. The prescriber writes a prescription. Two checkers inspect it. If they find a problem, it is written down and handed back, and the prescriber writes a *new* one knowing what to avoid. Repeats at most twice, then goes to the human doctor.

3 things that make it a real loop

1. **Feedback:** flags are injected into the next call as `loopContext` — so Agent 6 acts differently next time.
2. **Exit:** `if (both PASS) break` — ends as soon as it's safe.
3. **Bounded:** `MAX_LOOPS = 2` — a stubborn model can't loop forever.

Unresolved flags are saved to `flaggedDrugs`, shown to the doctor in a red panel, and **filtered out** before sending — an unsafe drug can never reach the patient.

Best line to say

"I bounded the loop and made the fallback a **human**, not an infinite retry." That is a safety answer — it shows you thought about what happens when the AI doesn't converge.

Why two separate checkers?

A model checking its own work leans toward approving it. Agents 7 and 8 are **independent** and grounded in outside databases — DDInter for interactions, RxClass for allergy classes. Same idea as a second doctor signing off.

Save After Every Step

Each agent's result is written to MongoDB the moment it finishes — not batched to the end.

- Live progress: the page polls the DB, so each save moves the bar.
- Refresh-safe: finished agents stay finished; only unfinished work restarts.
- Crash-safe: if the server dies at agent 7, agents 2–6 are already saved.
- Debuggable: you see exactly where it stopped.

LangGraph vs My Orchestrator

LangGraph idea	My MediIQ version
Node	An agent function (`AgentFn`).
Edge	Order of `await` lines in pipeline.ts.
Conditional edge	`if (a7 & a8 PASS) break;`
Cycle	The `for (loop < MAX_LOOPS)` block.
Shared state	`agentOutputs` + Appointment doc.
Checkpointer	`updateStep()` writes to MongoDB each step.
Recursion limit	`MAX_LOOPS = 2`.
Parallel branches	`Promise.allSettled([...])`
Graph picture	`AGENT_REGISTRY` + admin flow page.

What each choice gives you

LangGraph gives	Building it myself gives
Resume-after-crash free	Exact control of DB writes (dot-notation).
Streaming outputs free	Serverless correctness (awaited route + claim).
Graph picture free	No extra dependency in a big app.
Standard vocabulary	Full understanding of every failure mode.
Human-in-loop built in	Lighter, clearer for a small flow.

Balanced takeaway

Bigger graph, lots of branching, or resume-after-crash needed → a framework wins. For a mostly-linear 9-agent flow with one bounded loop, hand-writing was lighter and clearer. **Saying both sides is what makes the answer strong.**

All Formulae

Thing	Formula
Time in sequence	$T_{\text{seq}} = t_1 + t_2 + t_3$
Time in parallel	$T_{\text{par}} = \max(t_1, t_2, t_3)$
Speed-up	$S = T_{\text{seq}} / T_{\text{par}}$
Loop bound	loops $\leq$ `MAX_LOOPS` $= 2$

Don't Confuse These

ok vs flag vs error: ok = worked. flag = worked but found a problem. error = couldn't do it.

all vs allSettled: all = one fail kills all + throws away good work. allSettled = waits for all, keeps good work.

gather vs verify: gather = collect facts (1–5). verify = check it's safe (7,8).

Atomic claim vs State machine: claim = flip a flag so only one caller runs. state machine = one string tracking progress.

Node vs Edge: node = an agent. edge = the order of awaits.

Agents vs Orchestrator: agents = the brains/prompts. orchestrator = the pipes/rules.

Pre-warm vs Post-call: pre-warm = 3,4,5 during the call. post-call = 2,6,7,8,9 after.

Exam Quick Hits

- `pipeline.ts` = the orchestrator. Plain TS, no framework.
- 4 phases: gather → prescribe → verify → detect.
- Loop is bounded to 2; fallback = human doctor.
- Atomic claim = runs exactly once even with polling + 2 tabs.
- `allSettled` keeps good work when one agent fails.
- State in one DB field = refresh-safe + crash-visible.
- Vercel freezes on reply → await the pipeline + `maxDuration=300`.
- 2 checkers not self-check = no bias, grounded in DDInter + RxClass.
- Save after every step = live bar, crash-safe, debuggable.